Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №63

ТЕМА: Фізика анімації: відстеження зіткнень об'єктів (collision detection) та відскоки від меж полотна.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- ознайомитися з поняттям вектора швидкості об'єкта в анімації;
- навчитися реалізовувати відскок об'єкта від меж полотна;
- ознайомитися з базовим алгоритмом виявлення зіткнень (collision detection) прямокутних областей;
- розвинути навички побудови фізично правдоподібної анімації руху кількох об'єктів.

Теоретичні відомості

1. Вектор швидкості: dx і dy

У попередніх практичних роботах об'єкт рухався за жорстко заданим правилом (наприклад, завжди на 3 пікселі праворуч). Для реалістичнішого руху, зокрема для відскоків, зручно описувати рух через дві окремі змінні швидкості: dx (зміщення по осі x за один крок) і dy (зміщення по осі y за один крок). Такий запис називають вектором швидкості — він задає одночасно і напрямок, і величину руху.

```
ball = canvas.create_oval(190, 130, 220, 160, fill='#f2cc8f')
dx, dy = 4, 3   # рух по діагоналі: праворуч і вниз

def animate():
    canvas.move(ball, dx, dy)
    app.after(30, animate)

animate()
```

2. Відскок від вертикальних меж полотна

Щоб об'єкт «відбивався» від лівої чи правої межі полотна (а не виїжджав за них), на кожному кроці анімації перевіряють поточні координати об'єкта: якщо об'єкт досяг межі, знак швидкості dx змінюють на протилежний — це і є суть відскоку:

```
def animate():
    global dx, dy
    x1, y1, x2, y2 = canvas.coords(ball)
    if x1 <= 0 or x2 >= 380:
        dx = -dx
```

```
    canvas.move(ball, dx, dy)
    app.after(30, animate)
```

Вираз dx = -dx — стандартний прийом «розвороту» напрямку руху: якщо об'єкт рухався праворуч (dx додатне), після відскоку він почне рухатися ліворуч (dx стане від'ємним), і навпаки.

3. Відскок від усіх чотирьох меж (горизонтальних і вертикальних)

Аналогічну перевірку додають і для верхньої та нижньої меж полотна, змінюючи цього разу знак dy — у результаті об'єкт рухається по прямій, доки не досягне будь-якої з чотирьох меж, після чого змінює напрямок за законом «кут падіння дорівнює куту відбиття», подібно до руху більярдної кулі:

```
def animate():
    global dx, dy
    x1, y1, x2, y2 = canvas.coords(ball)
    if x1 <= 0 or x2 >= 380:
        dx = -dx
    if y1 <= 0 or y2 >= 280:
        dy = -dy
    canvas.move(ball, dx, dy)
    app.after(30, animate)
```

4. Виявлення зіткнення двох прямокутних областей

Зіткнення (collision detection) — це перевірка, чи перетинаються області двох об'єктів на екрані. Найпростіший і найпоширеніший спосіб — наближено описати кожен об'єкт (навіть якщо він круглий) прямокутною «обмежувальною рамкою» (bounding box) і перевірити, чи перетинаються дві такі рамки. Дві прямокутні області НЕ перетинаються, якщо одна з них цілком лежить ліворуч, праворуч, вище або нижче іншої; отже, вони перетинаються, якщо жодна з цих чотирьох умов не виконується:

```
def is_colliding(box1, box2):
    x1, y1, x2, y2 = box1
    x3, y3, x4, y4 = box2
    if x2 < x3 or x4 < x1 or y2 < y3 or y4 < y1:
        return False
    return True

box_a = canvas.coords(object_a)
box_b = canvas.coords(object_b)
if is_colliding(box_a, box_b):
    print('Зіткнення!')
```

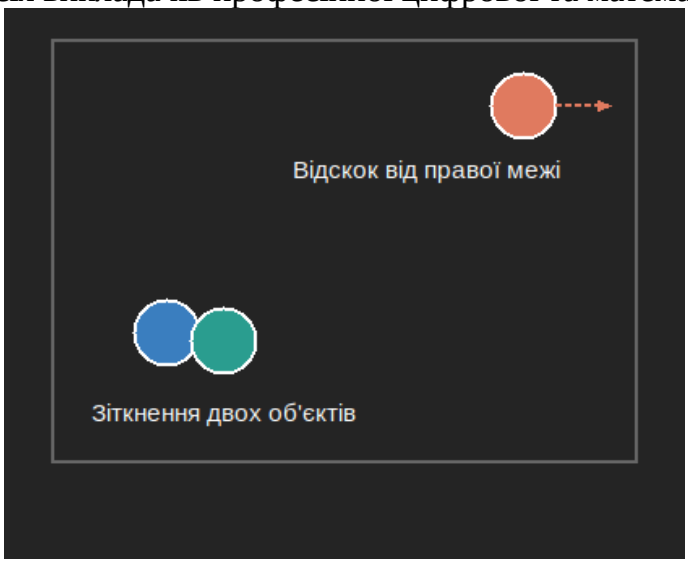


Рис. 1. Ілюстрація двох базових прийомів фізики анімації: відскок від межі полотна (згори) і зіткнення двох об'єктів (знизу)

Цей алгоритм є наближеним, оскільки перевіряє перетин прямокутних рамок, а не точних контурів фігур (наприклад, кіл) — для навчальних і більшості простих ігрових застосунків такої точності цілком достатньо, а точніша перевірка (наприклад, для кіл — за відстанню між центрами) розглядається як необов'язкове ускладнення.

5. Реакція на зіткнення

Що саме відбувається при виявленні зіткнення — залежить від задуму конкретної програми: об'єкти можуть зупинятися, відбиватися один від одного (обидва змінюють напрямок руху), один з об'єктів може зникати (canvas.delete()), або відбувається якась ігрова подія (нарахування очок, програш). У найпростішому варіанті достатньо, наприклад, змінити колір обох об'єктів або зупинити анімацію:

```
if is_colliding(canvas.coords(ball), canvas.coords(paddle)):
    canvas.itemconfig(ball, fill='red')
    return   # зупиняємо подальший рух у цьому кадрі
```

Хід роботи

Варіант і вимоги до здачі

a) N — ваш порядковий номер у списку групи (журнал). Швидкість залежить від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
1–6*	швидкість (dx, dy) = (3 + N mod 3, 2 + N mod 2)	(4, 3)

б) Кожне завдання — окремий файл Прізвище_ПР63_зN.py (наприклад, Ivanenko_PR63_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Рух будуйте через after(); перевірку зіткнення виконуйте функцією is_colliding(a, b), що повертає True/False.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР63_з1.py. На полотні 400×300 створіть коло радіусом 15 у точці (50, 50) і рухайте його на (dx, dy) пікселів кожні 20 мс.

   Приклад (N = 7): (dx, dy) = (4, 3); через 10 кроків центр (90, 80).

2. Додайте відскок від лівої й правої меж (зміна знаку dx).

   Приклад: після дотику до правої межі рух іде вліво.

3. Додайте відскок від усіх чотирьох меж (також dy).

   Приклад: траєкторія замкнена в межах полотна.

4. Додайте друге нерухоме коло в точці (200, 150) і функцію is_colliding(a, b), що за bbox перевіряє перетин; при зіткненні виводьте в консоль «Зіткнення!».

   Приклад: перетин рамок → «Зіткнення!»; без перетину — нічого.

5. Нехай перше коло рухається автоматично з відскоками, а друге керується стрілками (ПР60); перевіряйте зіткнення на кожному кроці та виводьте «Зіткнення!» один раз за одне зіткнення.

   Приклад: поки кола перетинаються, повідомлення не повторюється.

6. Завдання підвищеної складності*. Два рухомі кола відбиваються одне від одного: при зіткненні вони обмінюються векторами (dx1, dy1) ↔ (dx2, dy2).

Приклад: після зіткнення напрямки кіл змінилися, кола не «злипаються».

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, відскок лише від двох меж, зіткнення визначається за центрами замість рамок, повідомлення «Зіткнення!» повторюється кожен кадр, кола «злипаються») — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити швидкість чи правило відскоку). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Що таке вектор швидкості dx, dy і як він визначає напрямок і швидкість руху об'єкта?
2. Як реалізується відскок об'єкта від межі полотна на рівні коду?
3. За якими чотирма умовами визначають, що дві прямокутні області НЕ перетинаються?
4. Чому перевірку зіткнення кіл часто спрощують до перевірки перетину їхніх прямокутних обмежувальних рамок?
5. Наведіть приклад можливої реакції програми на виявлене зіткнення двох об'єктів.